Creating Products & Offers via the Fungies API

A developer's guide for the Fungies REST API. Verified live against the staging API on 2026-05-21.

A **product** on Fungies is a logical SKU; an **offer** is the priced, purchasable thing customers actually buy. You always create one product first, then one or more offers that point at it via `productId`. The hosted checkout URL is `<store>/checkout/<offer_id>`.

---

1. Pick the right environment

Env	Base URL	Get keys at
Staging	`https://api.stage.fungies.net/v0`	https://app.stage.fungies.net → Developers → API Keys
Production	`https://api.fungies.io/v0`	https://app.fungies.io → Developers → API Keys

Each environment is completely isolated — staging keys don't work against prod and vice versa. Always build against staging first.

2. Authentication

Every request needs two headers:

```
x-fngs-public-key: pub_...
x-fngs-secret-key: sec_...
Content-Type: application/json
```

Don't expose `sec_*` to the browser — it's the same as a password.

3. Create the product

`POST /v0/products/create`

```
{
  "name": "My Awesome Course",
  "type": "OneTimePayment",
  "status": "ACTIVE",
  "description": "Optional plain-text description.",
  "cover": "https://your-cdn.example.com/cover.jpg"
}
```

Field	Required	Notes
`name`	yes	Shown on the checkout page
`type`	yes	`OneTimePayment` is the only type the WP plugin supports today. Other valid values include `Subscription`.
`status`	yes	`ACTIVE` = visible/purchasable, `INACTIVE` = hidden
`description`	no	Strip HTML before sending — the API stores it as plain text
`cover`	no	Public HTTPS URL. Must be reachable from Fungies.

Response shape (the bit you care about):

```
{ "data": { "product": { "id": "85ce727c-48a7-4400-90ad-91bb52bcd4a6" } } }
```

Save `data.product.id` — you need it for the offer.

4. Create the offer

`POST /v0/offers/create`

```
{
  "name": "My Awesome Course - Full Access",
  "productId": "85ce727c-48a7-4400-90ad-91bb52bcd4a6",
  "currency": "USD",
  "price": 1999,
  "limit": null
}
```

Field	Required	Notes
`name`	yes	Shown in the cart line item
`productId`	yes	UUID from step 3
`currency`	yes	ISO 4217 uppercase. Must match your workspace currency — staging workspaces are usually USD.
`price`	yes	**Integer in minor units (cents).** `1999` = `$19.99`. Empirically the API also accepts decimals like `9.99` and converts them via ×100, but integer cents is the unambiguous form — stick with it.
`limit`	no	`null` = unlimited. Integer = max purchases before the offer is exhausted.

Response:

```
{ "data": { "offer": { "id": "370d144b-a2b0-4863-9ca2-beeabfc80cd6" } } }
```

5. Build the checkout URL

Once your Fungies store is **published** (Dashboard → Store → Publish), customers can buy the offer at:

```
https://<your-store-slug>.app.fungies.io/checkout/<offer_id>
```

Append custom fields and customer hints as query params to prefill the checkout and have them echoed back in the webhook:

```
?fngs-customer-email=alice@example.com
&fngs-customer-country=US
&user_id=u_12345         (custom field, surfaces in webhook payload)
&fngs-discount-code=SAVE10
```

6. End-to-end example (cURL)

```
PUB="pub_..."
SEC="sec_..."
BASE="https://api.stage.fungies.net/v0"

# 1. Create product
PRODUCT_ID=$(curl -sS -X POST "$BASE/products/create" \
  -H "x-fngs-public-key: $PUB" \
  -H "x-fngs-secret-key: $SEC" \
  -H "Content-Type: application/json" \
  -d '{"name":"My Course","type":"OneTimePayment","status":"ACTIVE"}' \
  | jq -r '.data.product.id')

# 2. Create offer pointed at it
OFFER_ID=$(curl -sS -X POST "$BASE/offers/create" \
  -H "x-fngs-public-key: $PUB" \
  -H "x-fngs-secret-key: $SEC" \
  -H "Content-Type: application/json" \
  -d "{\"name\":\"Full Access\",\"productId\":\"$PRODUCT_ID\",\"currency\":\"USD\",\"price\":1999,\"li
  | jq -r '.data.offer.id')

echo "Buy at: https://<store>.app.fungies.io/checkout/$OFFER_ID"
```

7. Updating instead of duplicating

The create endpoints **always create a new row**. To edit an existing product or offer, use:

- `PATCH /v0/products/{id}/update` — body must include `"id": "<same id>"` plus any fields to change
- `PATCH /v0/offers/{id}/update` — same convention

Same payload shape as create. The WP plugin's `Fungies_Product_Push` class is the canonical reference implementation for the create-or-update flow, including 404-recovery when an ID becomes stale after switching workspaces.

8. Things that will trip you up

1. **Currency mismatch** — if your workspace is USD and you send `currency: EUR`, the offer is rejected. Fetch a few existing offers (`GET /offers/list?take=10`) and inspect their `currency` to discover the workspace currency.
2. **Price units** — send integer cents. `19.99` works but is fragile.
3. **`status: ACTIVE`** — without this, the product is created but customers can't see or buy the offer.
4. **Store not published** — the `/checkout/<offer_id>` URL only works once the store is in published state in the Fungies dashboard.
5. **Custom-field validation** — if you've defined required custom fields on the product (e.g. `user_id`), checkout will refuse to load until they're passed as query params. See `help.fungies.io/for-saas-developers/using-customfields-to-parse-data-from-your-software-app`.

9. Verified live

This guide's flow was executed end-to-end against the **testworks staging workspace** on 2026-05-21:

```
[1] GET  /offers/list                     -> 200 OK, auth confirmed
[2] POST /products/create                 -> productId = 85ce727c-48a7-4400-90ad-91bb52bcd4a6
[3] POST /offers/create  (sent price=9.99) -> offerId   = 370d144b-a2b0-4863-9ca2-beeabfc80cd6
[4] GET  /offers/{id}    (round-trip)     -> price=999 USD   <- proves API stores cents
```
